

DATA CLEANING IN PYTHON

A Complete Teaching Guide

Regex • Duplicate Handling • Outlier Detection • Standardisation

Real-World Industry Examples | Hands-On Python Code | Trainer-Ready Format

Table of Contents

Table of Contents	2
Chapter 0: Why Data Cleaning Matters	3
0.1 The Cost of Dirty Data	3
0.2 The Four Pillars Covered in This Guide.....	3
0.3 Setup: Libraries & Sample Dataset	3
Chapter 1: Regex (Regular Expressions) for Data Cleaning	5
1.1 Regex Fundamentals — Quick Reference	5
1.2 Use Case 1 — Phone Number Extraction & Validation	5
Problem.....	5
Output & Explanation.....	6
1.3 Use Case 2 — Email Validation & Domain Extraction.....	7
1.4 Use Case 3 — Name Cleaning & Parsing	7
1.5 Use Case 4 — Date Parsing from Mixed Formats	8
1.6 Use Case 5 — Text Extraction: Pulling Amounts from Free Text	9
Chapter 2: Duplicate Detection & Handling	10
2.1 Exact Duplicate Detection	10
2.2 Fuzzy Duplicate Detection	10
City Canonicalisation Using Fuzzy Matching.....	11
2.3 Record Linkage — Cross-Dataset Deduplication.....	11
2.4 Handling Duplicates: Strategy Decision Matrix	12
Chapter 3: Outlier Detection	13
3.1 Method 1 — Z-Score (Parametric, assumes Normal distribution).....	13
3.2 Method 2 — IQR (Interquartile Range — Robust)	13
3.3 Method 3 — Isolation Forest (ML-Based, Multivariate)	14
3.4 Method 4 — Percentile Capping (Winsorisation)	14
3.5 Outlier Handling Strategy Guide.....	15
Chapter 4: Data Standardisation.....	16
4.1 Categorical Standardisation	16
4.2 Numeric Scaling for Machine Learning.....	16
4.3 Currency & Unit Standardisation	17
4.4 Missing Value Standardisation.....	18
Chapter 5: End-to-End Data Cleaning Pipeline	20
Chapter 6: Quick Reference Cheat Sheet.....	23
6.1 Regex Patterns for Common Data Fields	23
6.2 Outlier Method Comparison	23
6.3 Missing Value Decision Tree.....	23
6.4 Data Cleaning Checklist for Every Project.....	24

Chapter 0: Why Data Cleaning Matters

It is widely quoted in industry that data scientists and data engineers spend **70–80% of their time cleaning data** — and only 20–30% on actual analysis and modelling. Dirty data causes incorrect business decisions, failed ML models, regulatory fines, and customer-facing errors.

0.1 The Cost of Dirty Data

FACT IBM estimated that bad data costs the U.S. economy \$3.1 trillion per year (IBM, 2016). Even at the team level, one bad dataset can invalidate weeks of work.

Dirty Data Type	Real-World Impact	Business Risk
Phone numbers with mixed formats	CRM system rejects records	Sales pipeline gap
Duplicate customer records	Double-billing, wrong loyalty points	Revenue & legal risk
Extreme outliers in transaction data	ML fraud model trained on noise	False alerts, missed fraud
Inconsistent city / state names	Geo-reporting breaks	Wrong regional decisions
Missing values in loan applications	Scoring model crashes	Compliance failure

0.2 The Four Pillars Covered in This Guide

- **Regex (Regular Expressions)** – pattern-based text extraction and cleaning
- **Duplicate Handling** – identify, analyse, and resolve redundant records
- **Outlier Detection** – statistical and ML-based anomaly identification
- **Standardisation** – normalise formats, units, and scales for analysis

0.3 Setup: Libraries & Sample Dataset

Install the libraries used throughout this guide:

```
pip install pandas numpy scipy scikit-learn fuzzywuzzy python-Levenshtein phonenumbers
```

We will build a single synthetic dataset — an e-commerce customer order dataset — that runs through ALL four chapters:

```
import pandas as pd
import numpy as np
import re
```

```

from datetime import datetime

# — Synthetic E-Commerce Dataset —————
data = {
    "customer_id": [101, 102, 102, 103, 104, 105, 106, 107, 108, 109],
    "name": ["Anand Kumar", "PRIYA SHARMA", "Priya Sharma", " Ravi Gupta ",
            "Sunita-Patel", "John O'Brien", "meena iyer", " ", "Zhang Wei", "carlos
mendes"],
    "phone": ["9876543210", "+91-98765-43211", "98765 43211", "0091 9876543212",
            "(987) 654-3213", "987-654-3214", "9999999999", "123", "+1-800-555-
0100", "98765432"],
    "email": ["anand@gmail.com", "priya@yahoo.com", "priya@yahoo.com", "ravi.gupta@",
            "sunita@domain.co.in", "john.obrien@corp.com", "MEENA@GMAIL.COM",
            "", "zhangwei@outlook.com", "carlos@mail.com"],
    "city": ["Mumbai", "mumbai", "MUMBAI", "New Delhi", "Ahmedabad",
            "Bengaluru", "Bangalore", "Chennai", "Hyderabad", "Pune"],
    "order_amount": [1500.00, 2500.75, 2500.75, 980.00, 150000.00,
                    3200.50, 4100.25, -500.00, 1750.00, 2100.00],
    "order_date": ["2024-01-15", "15/01/2024", "Jan 15 2024", "2024-01-16",
                  "2024-01-17", "17-01-2024", "2024-01-18", "2024-01-19",
                  "2024-01-20", "20 Jan 2024"],
    "product_category": ["Electronics", "electronics", "Electronics ", "Clothing",
                        "Furniture", "furniture", "ELECTRONICS", "Clothing",
                        "Electronics", "Clothing"],
}

df = pd.DataFrame(data)
print(df.shape) # (10, 8)
print(df.dtypes)

```

Chapter 1: Regex (Regular Expressions) for Data Cleaning

A **regular expression** is a sequence of characters that defines a search pattern. In data cleaning, regex is the Swiss Army knife for extracting structured information from unstructured text fields such as phone numbers, emails, addresses, and free-text notes.

1.1 Regex Fundamentals — Quick Reference

Pattern	Meaning	Example Match
\d	Any digit (0–9)	9 in '9876'
\D	Any non-digit	a in 'a123'
\w	Word char (letter/digit/_)	A, 9, _
\W	Non-word character	@, #, space
\s	Whitespace	space, tab, newline
+	One or more	'\d+' matches '123'
*	Zero or more	'a*' matches '', 'a', 'aaa'
?	Zero or one	'colou?r' matches 'color' or 'colour'
{m,n}	Between m and n times	'\d{10}' matches exactly 10 digits
^	Start of string	'^\d' starts with digit
\$	End of string	'\d\$' ends with digit
[...]	Character class	'[aeiou]' any vowel
()	Capture group	'(\d{3})-(\d{4})'
	OR	'cat dog'

1.2 Use Case 1 — Phone Number Extraction & Validation

Industry: Telecom / Banking CRM — Customer data arrives from multiple sources with wildly different phone formats.

Problem

Our dataset contains phone numbers in at least 6 different formats. We need to: (a) extract the 10-digit number, (b) validate it, and (c) standardise to E.164 format (+91XXXXXXXXXX for India).

```
import re
```

```

phones = [
    "9876543210",          # plain 10-digit
    "+91-98765-43211",     # country code with dashes (11 digits after CC)
    "98765 43211",         # space-separated
    "0091 9876543212",     # 00-prefix country code
    "(987) 654-3213",      # US-style
    "987-654-3214",        # dashes
    "9999999999",          # valid 10-digit
    "123",                 # invalid (too short)
    "+1-800-555-0100",     # US number
    "98765432",           # 8 digits - invalid
]

def clean_phone(raw, default_country_code="91"):
    """
    Extract and standardise Indian phone numbers.
    Returns cleaned E.164 string or None if invalid.
    """
    if pd.isna(raw) or str(raw).strip() == "":
        return None

    # Step 1: Remove all non-digit characters
    digits_only = re.sub(r'\D', '', str(raw))

    # Step 2: Handle country-code prefixes
    if digits_only.startswith("91") and len(digits_only) == 12:
        digits_only = digits_only[2:]          # strip 91
    elif digits_only.startswith("0091") and len(digits_only) == 14:
        digits_only = digits_only[4:]          # strip 0091
    elif digits_only.startswith("1") and len(digits_only) == 11:
        # US number - keep as-is but flag separately
        return f"+{digits_only}"

    # Step 3: Validate - must be exactly 10 digits starting with 6/7/8/9
    pattern = r'^[6-9]\d{9}$'
    if re.match(pattern, digits_only):
        return f"+{default_country_code}{digits_only}"
    else:
        return None    # invalid

# Apply to dataset
df["phone_cleaned"] = df["phone"].apply(clean_phone)
df["phone_valid"] = df["phone_cleaned"].notna()

print(df[["phone", "phone_cleaned", "phone_valid"]].to_string())

```

Output & Explanation

Raw Input	Cleaned Output	Valid?	Rule Applied
9876543210	+919876543210	Yes	10-digit, starts with 9
+91-98765-43211	None	No	11 digits after CC stripped
0091 9876543212	+919876543212	Yes	0091 prefix removed
123	None	No	Too short
+1-800-555-0100	+18005550100	Yes (US)	US CC detected

1.3 Use Case 2 — Email Validation & Domain Extraction

Industry: Marketing / SaaS — Email lists from lead-gen campaigns are notoriously messy.

```
emails = [
    "anand@gmail.com",      # valid
    "priya@yahoo.com",     # valid
    "ravi.gupta@",          # missing domain
    "MEENA@GMAIL.COM",     # uppercase - valid but normalise
    "",                     # empty
    "carlos@@mail.com",    # double @
    "sunita@domain.co.in", # valid .co.in
    "john.obrien@corp.com", # valid
]

# RFC 5322-inspired (simplified) regex
EMAIL_REGEX = re.compile(
    r'^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$'
)

def validate_email(email):
    if pd.isna(email) or str(email).strip() == "":
        return None, False, None

    email = str(email).strip().lower()      # normalise case

    # Check for double @ (quick guard)
    if email.count('@') != 1:
        return email, False, None

    is_valid = bool(EMAIL_REGEX.match(email))

    # Extract domain if valid
    domain = email.split('@')[1] if is_valid else None
    return email, is_valid, domain

df[["email_clean", "email_valid", "email_domain"]] = df["email"].apply(
    lambda e: pd.Series(validate_email(e))
)

# Segmentation: B2C vs B2B
PERSONAL_DOMAINS = {"gmail.com", "yahoo.com", "hotmail.com", "outlook.com"}
df["email_type"] = df["email_domain"].apply(
    lambda d: "B2C" if d in PERSONAL_DOMAINS else ("B2B" if pd.notna(d) else "Unknown")
)

print(df[["email", "email_clean", "email_valid", "email_type"]].to_string())
```

1.4 Use Case 3 — Name Cleaning & Parsing

Industry: Banking KYC / HR Systems — Names arrive in inconsistent capitalisation, extra whitespace, and joined formats.

```
import re

def clean_name(raw):
    if pd.isna(raw) or str(raw).strip() == "":
        return None, None, None
```

```

name = str(raw).strip()

# Remove multiple spaces
name = re.sub(r'\s+', ' ', name)

# Title-case: handle hyphenated names correctly
name = re.sub(r"[A-Za-z]+(['-][A-Za-z]+)*",
              lambda m: m.group().title(), name)

# Split into first / last
parts = name.split()
first = parts[0] if len(parts) >= 1 else None
last = parts[-1] if len(parts) >= 2 else None

return name, first, last

df[["name_clean", "first_name", "last_name"]] = df["name"].apply(
    lambda n: pd.Series(clean_name(n))
)
print(df[["name", "name_clean", "first_name", "last_name"]].to_string())

```

1.5 Use Case 4 — Date Parsing from Mixed Formats

Industry: Supply Chain / Finance — Reports from legacy systems mix DD/MM/YYYY, YYYY-MM-DD, 'Jan 15 2024', etc.

```

from dateutil import parser as dateparser

DATE_PATTERNS = [
    (r'^\d{4}-\d{2}-\d{2}$',      '%Y-%m-%d'),    # 2024-01-15
    (r'^\d{2}/\d{2}/\d{4}$',     '%d/%m/%Y'),     # 15/01/2024
    (r'^\d{2}-\d{2}-\d{4}$',     '%d-%m-%Y'),     # 17-01-2024
]

def parse_date(raw):
    if pd.isna(raw):
        return None, "missing"

    raw = str(raw).strip()

    # Try pattern-matched parsing first (faster, unambiguous)
    for pattern, fmt in DATE_PATTERNS:
        if re.match(pattern, raw):
            try:
                return datetime.strptime(raw, fmt).date(), "regex_matched"
            except ValueError:
                pass

    # Fallback: dateutil fuzzy parser
    try:
        return dateparser.parse(raw, dayfirst=True).date(), "fuzzy_parsed"
    except Exception:
        return None, "parse_error"

df[["order_date_clean", "date_parse_method"]] = df["order_date"].apply(
    lambda d: pd.Series(parse_date(d))
)
print(df[["order_date", "order_date_clean", "date_parse_method"]].to_string())

```


1.6 Use Case 5 — Text Extraction: Pulling Amounts from Free Text

Industry: Insurance Claims / Procurement — Notes fields contain amounts buried in sentences.

```
claim_notes = [
    "Claim approved for Rs. 45,000 after deductible",
    "Reimbursement of INR 1,23,456.78 processed",
    "Amount: $2,500 USD covered",
    "No claim amount mentioned",
    "Paid ₹78000 to hospital",
]

def extract_amount(text):
    """Extract first numeric amount from text, regardless of currency symbol."""
    if not text:
        return None
    # Match optional currency symbol, then digits with commas/periods
    pattern = r'(?Rs\.?|INR|\$|₹)?\s*([\d,]+(?:\.\d{1,2})?)'
    match = re.search(pattern, text, re.IGNORECASE)
    if match:
        # Remove commas (Indian/international number format)
        amount_str = match.group(1).replace(',', '')
        return float(amount_str)
    return None

for note in claim_notes:
    amt = extract_amount(note)
    print(f" Input : {note}")
    print(f" Amount: {amt}\n")
```

Chapter 2: Duplicate Detection & Handling

Duplicates are **silent killers** in analytics. A customer counted twice inflates churn metrics; a transaction recorded twice doubles revenue. There are two main types: **exact duplicates** (identical rows) and **fuzzy duplicates** (same entity, slightly different representations).

2.1 Exact Duplicate Detection

Industry: Retail / Banking — Same order submitted twice due to double-click or network retry.

```
# — 1. Detect fully identical rows —————
exact_dupes = df[df.duplicated(keep=False)]
print(f"Exact duplicate rows: {len(exact_dupes)}")
print(exact_dupes[["customer_id", "name", "order_amount"]])

# — 2. Detect key-level duplicates —————
# Identify which rows share same customer_id + order_amount (likely double-submit)
key_dupes = df[df.duplicated(subset=["customer_id", "order_amount"], keep=False)]
print(f"\nKey-level duplicates: {len(key_dupes)}")

# — 3. Remove duplicates: keep first occurrence —————
df_deduped = df.drop_duplicates(subset=["customer_id", "order_amount"], keep="first")
print(f"\nRows before: {len(df)} | After dedup: {len(df_deduped)}")

# — 4. Audit trail: tag duplicates before removing —————
df["is_duplicate"] = df.duplicated(subset=["customer_id", "order_amount"],
                                   keep="first")
df["duplicate_of_id"] = df.apply(
    lambda row: df[
        df.duplicated(subset=["customer_id", "order_amount"], keep="last") &
        (df["customer_id"] == row["customer_id"])
    ].index.tolist() if row["is_duplicate"] else None,
    axis=1
)
print(df[["customer_id", "name", "order_amount", "is_duplicate"]])
```

2.2 Fuzzy Duplicate Detection

Industry: Hospitality / Healthcare — 'Mumbai', 'mumbai', 'MUMBAI', 'Bombay' are all the same city but differ as strings. Same for patient names: 'Ravi Gupta' vs 'R. Gupta'.

```
from fuzzywuzzy import fuzz, process
import itertools

# — Fuzzy name matching —————
names = df["name_clean"].dropna().tolist()

def find_fuzzy_dupes(name_list, threshold=85):
    """
    Compare all pairs and flag likely duplicates above threshold.
    Uses token_sort_ratio to handle word-order differences.
    Returns list of (name_a, name_b, score) tuples.
    """
    dupes = []
```

```

    for a, b in itertools.combinations(name_list, 2):
        score = fuzz.token_sort_ratio(a, b)
        if score >= threshold:
            dupes.append((a, b, score))
    return dupes

fuzzy_pairs = find_fuzzy_dupes(names, threshold=80)
for pair in fuzzy_pairs:
    print(f" '{pair[0]}' <-> '{pair[1]}' | Score: {pair[2]}")

```

City Canonicalisation Using Fuzzy Matching

```

# — Canonical city names list —————
CANONICAL_CITIES = [
    "Mumbai", "New Delhi", "Ahmedabad", "Bengaluru",
    "Chennai", "Hyderabad", "Pune", "Kolkata"
]

def standardise_city(raw_city):
    if pd.isna(raw_city) or str(raw_city).strip() == "":
        return None, 0

    raw = str(raw_city).strip()

    # First try exact case-insensitive match
    for city in CANONICAL_CITIES:
        if raw.lower() == city.lower():
            return city, 100

    # Fuzzy fallback
    best_match, score = process.extractOne(raw, CANONICAL_CITIES,
    scorer=fuzz.token_sort_ratio)
    return (best_match, score) if score >= 75 else (raw, score)

df[["city_std", "city_match_score"]] = df["city"].apply(
    lambda c: pd.Series(standardise_city(c))
)

print(df[["city", "city_std", "city_match_score"]].to_string())

# — Note: 'Bangalore' and 'Bengaluru' will map to 'Bengaluru' —————

```

2.3 Record Linkage — Cross-Dataset Deduplication

Industry: Government / Insurance — Two databases (CRM and billing) must be matched without a common unique key.

```

# Simulate two datasets that should be the same person
crm_df = pd.DataFrame({
    "name": ["Anand Kumar", "Priya Sharma", "Ravi Gupta"],
    "phone": ["+919876543210", "+919876543211", "+919876543212"],
    "email": ["anand@gmail.com", "priya@yahoo.com", "ravi@corp.com"],
})

billing_df = pd.DataFrame({
    "full_name": ["ANAND KUMAR", "P. Sharma", "Ravi Gupta"],
    "mobile": ["9876543210", "98765 43211", "91-9876543212"],
    "mail": ["anand@gmail.com", "priya@yahoo.com", "ravi@corp.com"],
})

```

```

# Normalise keys in both datasets
def normalise_name(n):
    return re.sub(r'\s+', ' ', str(n)).strip().lower()

crm_df["name_key"] = crm_df["name"].apply(normalise_name)
billing_df["name_key"] = billing_df["full_name"].apply(normalise_name)

# Strategy: score on 3 fields and link if total >= 2.0
def match_score(row_a, row_b):
    name_score = fuzz.token_sort_ratio(row_a["name_key"], row_b["name_key"]) / 100
    email_score = 1.0 if row_a["email"] == row_b["mail"] else 0.0
    return name_score * 0.5 + email_score * 0.5

matches = []
for _, r_crm in crm_df.iterrows():
    for _, r_bill in billing_df.iterrows():
        score = match_score(r_crm, r_bill)
        if score >= 0.7:
            matches.append({"crm_name": r_crm["name"],
                           "billing_name": r_bill["full_name"],
                           "match_score": round(score, 2)})

print(pd.DataFrame(matches))

```

2.4 Handling Duplicates: Strategy Decision Matrix

Scenario	Best Strategy	pandas Function
Identical rows (ETL reload)	Keep first, drop rest	<code>drop_duplicates(keep='first')</code>
Same person, slight name diff	Fuzzy match + manual review queue	<code>fuzzywuzzy process.extractOne</code>
Same transaction, different timestamps	Keep latest, discard older	<code>sort + drop_duplicates(keep='last')</code>
Cross-system same entity	Record linkage, create master_id	Custom scoring function
Uncertain — need business sign-off	Flag, don't delete — audit trail	Add <code>is_duplicate</code> column

Chapter 3: Outlier Detection

An **outlier** is a data point that differs significantly from other observations. It can be: a **data error** (negative order amount), a **genuine extreme** (a high-value customer), or a **fraud signal** (unusually large transaction).

KEY INSIGHT Always ask WHY before removing an outlier. In fraud detection, the outlier IS the signal. In sensor data, it may be a faulty reading. Context is everything.

3.1 Method 1 — Z-Score (Parametric, assumes Normal distribution)

Industry: Retail / Finance — Detecting anomalous transaction amounts in a bank's daily settlement file.

```
from scipy import stats
import numpy as np

# — Apply to order_amount —————
df["amount_zscore"] = stats.zscore(df["order_amount"])

# Threshold: |z| > 2.5 is an outlier (stricter than the common 3.0)
df["outlier_zscore"] = df["amount_zscore"].abs() > 2.5

print("Outliers detected by Z-Score:")
print(df[df["outlier_zscore"]][["customer_id", "name", "order_amount",
"amount_zscore"]])

# — Visual inspection —————
# df["order_amount"].plot(kind='box', title='Order Amount Distribution')
# plt.show()

# — When to use Z-Score —————
# ✓ Data is approximately normally distributed
# ✓ No extreme skew
# ✗ Avoid for small datasets (n < 30)
# ✗ Avoid if data is heavily skewed (use IQR instead)
```

3.2 Method 2 — IQR (Interquartile Range — Robust)

Industry: E-Commerce — Order amounts are right-skewed (a few very large orders). IQR is robust to skewness.

```
def detect_outliers_iqr(series, multiplier=1.5):
    """
    Standard IQR outlier detection.
    multiplier=1.5 → mild outliers
    multiplier=3.0 → extreme outliers only
    """
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
```

```

lower_fence = Q1 - multiplier * IQR
upper_fence = Q3 + multiplier * IQR

return (series < lower_fence) | (series > upper_fence), lower_fence, upper_fence

df["outlier_iqr"], lower, upper = detect_outliers_iqr(df["order_amount"])

print(f"IQR Fences → Lower: {lower:.2f} | Upper: {upper:.2f}")
print("\nOutliers detected by IQR:")
print(df[df["outlier_iqr"]][["customer_id", "name", "order_amount"]])

# — Business Logic Outliers —————
# Orders must be positive and ≤ ₹1,00,000 (business rule)
df["outlier_business_rule"] = (df["order_amount"] <= 0) | (df["order_amount"] >
100000)
print("\nBusiness Rule Violations:")
print(df[df["outlier_business_rule"]][["customer_id", "name", "order_amount"]])

```

3.3 Method 3 — Isolation Forest (ML-Based, Multivariate)

Industry: Fraud Detection / Cybersecurity — Detect anomalies across multiple features simultaneously (amount + frequency + time).

```

from sklearn.ensemble import IsolationForest

# — Simulate richer dataset for multi-feature detection —————
np.random.seed(42)
n = 500
transactions = pd.DataFrame({
    "amount": np.concatenate([np.random.normal(2000, 500, 490),
                               np.random.uniform(50000, 200000, 10)]),
    "frequency": np.concatenate([np.random.randint(1, 10, 490),
                                 np.random.randint(50, 200, 10)]),
    "hour_of_day": np.concatenate([np.random.randint(8, 22, 490),
                                   np.random.randint(1, 4, 10)]), # late-night
    "anomalies": np.zeros(n)
})

# — Train Isolation Forest —————
iso = IsolationForest(
    n_estimators=100,
    contamination=0.05, # expect 5% anomalies
    random_state=42
)
transactions["anomaly_score"] = iso.fit_predict(
    transactions[["amount", "frequency", "hour_of_day"]]
)
# -1 = anomaly, 1 = normal
transactions["is_anomaly"] = transactions["anomaly_score"] == -1

anomalies = transactions[transactions["is_anomaly"]]
print(f"Total transactions: {len(transactions)}")
print(f"Anomalies detected: {len(anomalies)}")
print(f"({len(anomalies)/len(transactions)*100:.1f}%)")
print(anomalies.describe())

```

3.4 Method 4 — Percentile Capping (Winsorisation)

Industry: Credit Scoring / Insurance — Instead of removing outliers, cap them at the 1st/99th percentile to retain the record but reduce distortion.

```
def winsorise(series, lower_pct=0.01, upper_pct=0.99):
    """
    Cap extreme values at percentile boundaries.
    Preferred when:
    - Outliers are real but extreme (not errors)
    - You cannot afford to lose records (small dataset)
    - The model is sensitive to scale (linear regression)
    """
    lower_bound = series.quantile(lower_pct)
    upper_bound = series.quantile(upper_pct)

    capped = series.clip(lower=lower_bound, upper=upper_bound)

    n_capped_lower = (series < lower_bound).sum()
    n_capped_upper = (series > upper_bound).sum()

    print(f"Capped {n_capped_lower} values below {lower_bound:.2f}")
    print(f"Capped {n_capped_upper} values above {upper_bound:.2f}")
    return capped

df["order_amount_winsorised"] = winsorise(df["order_amount"])
print(df[["order_amount", "order_amount_winsorised"]].to_string())
```

3.5 Outlier Handling Strategy Guide

Type of Outlier	Action	Rationale
Data entry error (negative age)	Correct or remove	Clear mistake — no signal value
Measurement error (sensor spike)	Remove or interpolate	Not a real data point
Genuine extreme (₹10L order from VIP)	Keep, flag, segment separately	Real business event
Fraud/anomaly signal	Keep, escalate to fraud team	The outlier IS the insight
Ambiguous (could be either)	Cap (Winsorise)	Conservative — keep the record

Chapter 4: Data Standardisation

Standardisation means bringing data into a **consistent, comparable format**. This covers: (a) format standardisation (dates, phones, categories), (b) value normalisation (Min-Max, Z-score scaling for ML), and (c) unit harmonisation (currencies, units of measure).

4.1 Categorical Standardisation

Industry: Retail Analytics — Product category fields from multiple data entry operators: 'electronics', 'Electronics', 'ELECTRONICS' must all map to 'Electronics'.

```
# — Simple case + strip standardisation —————
df["category_std"] = (
    df["product_category"]
    .str.strip()          # remove leading/trailing spaces
    .str.title()          # Title Case: 'ELECTRONICS' → 'Electronics'
)

print(df[["product_category", "category_std"]].drop_duplicates())

# — Mapping dictionary for domain-specific synonyms —————
CATEGORY_MAP = {
    "electronics": "Electronics",
    "electronic": "Electronics",
    "clothing": "Apparel",
    "clothes": "Apparel",
    "furniture": "Home & Furniture",
    "home": "Home & Furniture",
}

def standardise_category(raw):
    if pd.isna(raw):
        return "Unknown"
    key = str(raw).strip().lower()
    return CATEGORY_MAP.get(key, str(raw).strip().title())

df["category_mapped"] = df["product_category"].apply(standardise_category)
print(df[["product_category", "category_mapped"]].drop_duplicates())
```

4.2 Numeric Scaling for Machine Learning

Industry: ML / Data Science — When training a model, features on different scales (age 0–100, salary 0–10,00,000) must be normalised so no single feature dominates.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

# — Sample feature dataframe —————
features = pd.DataFrame({
    "age": [25, 34, 45, 28, 52, 61, 29, 38, 47, 33],
    "annual_income": [350000, 850000, 1200000, 480000, 950000,
                     1500000, 420000, 780000, 1100000, 620000],
    "credit_score": [650, 720, 810, 680, 750, 790, 700, 740, 800, 710],
})
```



```

# — 1. Min-Max Scaling (0 to 1) —————
# Use when: neural networks, image data, distance-based models (KNN)
mm_scaler = MinMaxScaler()
features_minmax = pd.DataFrame(
    mm_scaler.fit_transform(features),
    columns=[f"{c}_minmax" for c in features.columns]
)

# — 2. Z-Score / Standard Scaling (mean=0, std=1) —————
# Use when: linear regression, logistic regression, SVM, PCA
std_scaler = StandardScaler()
features_std = pd.DataFrame(
    std_scaler.fit_transform(features),
    columns=[f"{c}_std" for c in features.columns]
)

# — 3. Robust Scaling (uses IQR — handles outliers) —————
# Use when: data has outliers, tree-based models
rob_scaler = RobustScaler()
features_robust = pd.DataFrame(
    rob_scaler.fit_transform(features),
    columns=[f"{c}_robust" for c in features.columns]
)

result = pd.concat([features, features_minmax, features_std, features_robust], axis=1)
print(result[["age", "age_minmax", "age_std", "age_robust"]].round(3))

# — IMPORTANT: Always fit on TRAIN, transform on TRAIN + TEST —————
# from sklearn.model_selection import train_test_split
# X_train, X_test = train_test_split(features, test_size=0.2)
# scaler.fit(X_train)
# X_train_scaled = scaler.transform(X_train)
# X_test_scaled = scaler.transform(X_test) # ← use train's fit, not refit

```

Scaler	Formula	Best For	Sensitive to Outliers?
MinMaxScaler	$(x - \min) / (\max - \min)$	Neural nets, KNN, image data	Yes — outliers shift range
StandardScaler	$(x - \text{mean}) / \text{std}$	Linear/logistic regression, PCA	Moderate
RobustScaler	$(x - \text{median}) / \text{IQR}$	Data with outliers, general ML	No — IQR-based
Log Transform	$\log(x+1)$	Right-skewed data (salaries)	Reduces outlier impact

4.3 Currency & Unit Standardisation

Industry: International E-Commerce / Finance — Sales data arrives in USD, EUR, INR, GBP from different regional systems.

```

# — Currency normalisation to INR —————
# In production, use a live FX API. Here we use static rates for demo.
FX_RATES_TO_INR = {
    "INR": 1.0,
    "USD": 83.5,
    "EUR": 90.2,
}

```

```

    "GBP": 105.0,
    "AED": 22.7,
}

transactions_multi = pd.DataFrame({
    "sale_id": [1, 2, 3, 4, 5],
    "amount": [1500, 250, 200, 1000, 5000],
    "currency": ["INR", "USD", "EUR", "AED", "INR"],
    "region": ["India", "USA", "Germany", "UAE", "India"],
})

def convert_to_inr(row):
    rate = FX_RATES_TO_INR.get(row["currency"], None)
    if rate is None:
        return None, f"Unknown currency: {row['currency']}"
    return round(row["amount"] * rate, 2), "OK"

transactions_multi[["amount_inr", "fx_status"]] = transactions_multi.apply(
    lambda r: pd.Series(convert_to_inr(r)), axis=1
)

print(transactions_multi.to_string())

```

4.4 Missing Value Standardisation

Industry: Healthcare / Surveys — Missing values encoded inconsistently: -999, 'N/A', 'none', "", NULL.

```

import numpy as np

# — Standardise all missing-value representations to NaN —————
MISSING_MARKERS = [-999, -1, "N/A", "NA", "n/a", "none", "None", "NONE",
                  "null", "NULL", "nan", "NaN", "unknown", "", " "]

def standardise_nulls(df):
    """Replace all common 'missing' markers with np.nan."""
    return df.replace(MISSING_MARKERS, np.nan)

# — Missing value treatment strategies —————
def fill_missing(df, strategy="smart"):
    df = df.copy()
    for col in df.columns:
        null_pct = df[col].isna().mean() * 100

        if null_pct == 0:
            continue
        elif null_pct > 60:
            print(f"DROP column '{col}' - {null_pct:.1f}% missing")
            df.drop(columns=[col], inplace=True)
        elif df[col].dtype in ["float64", "int64"]:
            # Numeric: use median (robust to outliers)
            median_val = df[col].median()
            df[col].fillna(median_val, inplace=True)
            print(f"FILLED '{col}' with median {median_val} ({null_pct:.1f}% missing)")
        else:
            # Categorical: use mode (most frequent)
            mode_val = df[col].mode()[0] if not df[col].mode().empty else "Unknown"
            df[col].fillna(mode_val, inplace=True)
            print(f"FILLED '{col}' with mode '{mode_val}' ({null_pct:.1f}% missing)")
    return df

```

```
df_clean = standardise_nulls(df)
df_clean = fill_missing(df_clean)
```

Chapter 5: End-to-End Data Cleaning Pipeline

Industry: E-Commerce Analytics Platform — Combine all four chapters into a single, production-ready cleaning class. This is the pattern used in real enterprise data engineering projects.

```
import pandas as pd
import numpy as np
import re
from scipy import stats
from sklearn.preprocessing import RobustScaler
from fuzzywuzzy import process, fuzz

class DataCleaner:
    """
    Production-grade data cleaning pipeline for e-commerce customer/order data.
    Each method returns self for method-chaining.
    """

    CANONICAL_CITIES = ["Mumbai", "New Delhi", "Ahmedabad", "Bengaluru",
                        "Chennai", "Hyderabad", "Pune", "Kolkata"]
    MISSING_MARKERS = [-999, -1, "N/A", "NA", "none", "NULL", "", " "]
    EMAIL_REGEX = re.compile(r'^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$')

    def __init__(self, df):
        self.df = df.copy()
        self.log = [] # audit trail

    def _log(self, step, detail):
        self.log.append({"step": step, "detail": detail})
        print(f" [✓] {step}: {detail}")

    # — Step 1: Null standardisation —————
    def standardise_nulls(self):
        before = self.df.isna().sum().sum()
        self.df.replace(self.MISSING_MARKERS, np.nan, inplace=True)
        after = self.df.isna().sum().sum()
        self._log("NULL_STD", f"NaN count: {before} → {after}")
        return self

    # — Step 2: Name cleaning —————
    def clean_names(self):
        def _clean(row):
            if pd.isna(row): return None
            n = re.sub(r'\s+', ' ', str(row).strip())
            return re.sub(r"[A-Za-z]+(['-][A-Za-z]+)*",
                        lambda m: m.group().title(), n)
        self.df["name_clean"] = self.df["name"].apply(_clean)
        self._log("NAME_CLEAN", f"{self.df['name_clean'].notna().sum()} names cleaned")
        return self

    # — Step 3: Phone cleaning —————
    def clean_phones(self):
        def _clean(row):
            if pd.isna(row): return None
            digits = re.sub(r'\D', '', str(row))
            if digits.startswith("91") and len(digits) == 12:
                digits = digits[2:]
            return digits
```

```

        elif digits.startswith("0091") and len(digits) == 14:
            digits = digits[4:]
            return f"+91{digits}" if re.match(r'^[6-9]\d{9}$', digits) else None
        self.df["phone_clean"] = self.df["phone"].apply(_clean)
        valid = self.df["phone_clean"].notna().sum()
        self._log("PHONE_CLEAN", f"{valid}/{len(self.df)} phones valid")
        return self

# — Step 4: Email cleaning —————
def clean_emails(self):
    def _clean(row):
        if pd.isna(row) or str(row).count('@') != 1: return None
        e = str(row).strip().lower()
        return e if self.EMAIL_REGEX.match(e) else None
    self.df["email_clean"] = self.df["email"].apply(_clean)
    valid = self.df["email_clean"].notna().sum()
    self._log("EMAIL_CLEAN", f"{valid}/{len(self.df)} emails valid")
    return self

# — Step 5: City standardisation —————
def standardise_cities(self):
    def _std(row):
        if pd.isna(row): return None
        match, score = process.extractOne(
            str(row).strip(), self.CANONICAL_CITIES, scorer=fuzz.token_sort_ratio)
        return match if score >= 75 else str(row).strip().title()
    self.df["city_std"] = self.df["city"].apply(_std)
    self._log("CITY_STD", "Cities mapped to canonical list")
    return self

# — Step 6: Deduplication —————
def remove_duplicates(self, subset=None):
    before = len(self.df)
    self.df["is_duplicate"] = self.df.duplicated(subset=subset, keep="first")
    self.df = self.df[~self.df["is_duplicate"]].reset_index(drop=True)
    after = len(self.df)
    self._log("DEDUP", f"Removed {before - after} duplicates | {after} rows remain")
    return self

# — Step 7: Outlier handling —————
def handle_outliers(self, col, method="iqr", action="flag"):
    series = self.df[col]
    if method == "iqr":
        Q1, Q3 = series.quantile(0.25), series.quantile(0.75)
        IQR = Q3 - Q1
        mask = (series < Q1 - 1.5 * IQR) | (series > Q3 + 1.5 * IQR)
    elif method == "zscore":
        mask = stats.zscore(series).abs() > 2.5
    elif method == "business":
        mask = (series <= 0) | (series > 100000)

    self.df[f"{col}_outlier"] = mask
    if action == "cap":
        lower = series.quantile(0.01)
        upper = series.quantile(0.99)
        self.df[col] = series.clip(lower, upper)
        self._log("OUTLIER_CAP", f"{col}: {mask.sum()} capped")
    else:
        self._log("OUTLIER_FLAG", f"{col}: {mask.sum()} flagged")
    return self

# — Step 8: Category standardisation —————
def standardise_categories(self, col, mapping=None):
    self.df[f"{col}_std"] = self.df[col].str.strip().str.title()

```

```
        if mapping:
            self.df[f"{col}_std"] = self.df[f"{col}_std"].replace(mapping)
        self.log("CAT_STD", f"{col} standardised")
        return self

# — Run full pipeline —————
def run(self):
    return (self
            .standardise_nulls()
            .clean_names()
            .clean_phones()
            .clean_emails()
            .standardise_cities()
            .remove_duplicates(subset=["customer_id", "order_amount"])
            .handle_outliers("order_amount", method="business", action="flag")
            .standardise_categories("product_category"))

def summary(self):
    print("\n" + "="*60)
    print("CLEANING PIPELINE SUMMARY")
    print("="*60)
    for entry in self.log:
        print(f"  {entry['step']:<20} {entry['detail']}")
    print(f"\nFinal shape: {self.df.shape}")
    print(f"Null %: {self.df.isna().mean().mean()*100:.1f}%")
    return self.df

# — Run it —————
cleaner = DataCleaner(df)
df_final = cleaner.run().summary()
```

Chapter 6: Quick Reference Cheat Sheet

6.1 Regex Patterns for Common Data Fields

Field	Regex Pattern	Notes
Indian Phone	<code>^[6-9]\d{9}\$</code>	10-digit, starts with 6/7/8/9
Email (simple)	<code>^[w.%+-]+@[w.-]+\.[a-zA-Z]{2,}\$</code>	Most common email formats
PAN Card	<code>^[A-Z]{5}[0-9]{4}[A-Z]{1}\$</code>	Indian PAN number
Aadhar	<code>^[2-9]{1}[0-9]{11}\$</code>	12 digits, starts with 2–9
Pincode	<code>^[1-9][0-9]{5}\$</code>	6-digit Indian pincode
GST Number	<code>^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}\$</code>	15-char GST format
Date YYYY-MM-DD	<code>^\d{4}-(0[1-9] 1[0-2])-(0[1-9] 1[2]\d 3[01])\$</code>	ISO date
Currency Amount	<code>[\d,]+(?:\.\d{1,2})?</code>	With optional decimals

6.2 Outlier Method Comparison

Method	Type	Use When	Limitation
Z-Score	Parametric	Normal distribution	Fails on skewed data
IQR	Non-parametric	Skewed data, general use	May miss multivariate anomalies
Isolation Forest	ML-based	High-dimensional, fraud detection	Needs tuning, black-box
Winsorisation	Capping	Keep all records, reduce extremes	Distorts true values slightly
Business Rules	Domain	Known valid ranges	Requires domain expertise

6.3 Missing Value Decision Tree

```
# Decision guide for missing value handling:
#
# 1. > 60% missing?
#     └─ Consider DROPPING the column
#
# 2. < 5% missing + numeric?
#     └─ IMPUTE with MEDIAN (robust) or MEAN (if symmetric)
```

```
#  
# 3. < 5% missing + categorical?  
#     └ IMPUTE with MODE or "Unknown"  
#  
# 4. Temporal data (time series)?  
#     └ FORWARD FILL or interpolation  
#  
# 5. Data Missing Not at Random (MNAR)?  
#     └ Create binary indicator column (is_missing flag)  
#     └ May need domain-specific imputation or leave as-is  
#  
# 6. For ML models: use IterativeImputer or KNNImputer  
#     from sklearn.impute import IterativeImputer, KNNImputer
```

6.4 Data Cleaning Checklist for Every Project

1. Profile the data first — use `df.info()`, `df.describe()`, `df.isna().sum()`
2. Standardise nulls — map all missing markers to `np.nan`
3. Fix data types — dates as `datetime`, IDs as `string`, amounts as `float`
4. Validate and clean text fields — phone, email, name using `regex`
5. Detect and resolve duplicates — exact then fuzzy
6. Detect outliers — choose method based on distribution shape
7. Decide action for each outlier — keep, cap, flag, or remove
8. Standardise categorical values — case, synonyms, canonical mapping
9. Scale numeric features — only before ML, never before EDA
10. Build an audit log — record every transformation with before/after counts